PRD & Technical Specification v1.0: daraja-core & cdk-daraja-processor

Status: v1.0 (Verified Specification)
Date: October 2026
License: MIT / Apache-2.0
Core Runtime: Rust (Edition 2024 — `tokio`, `axum`, `tonic`, `sqlx`)
External Integration Boundary: Language-Agnostic (gRPC / Protobuf & REST Sidecar + `AGENTS.md` Coding Agent Skill for TypeScript, Python, Go, C#, PHP, and Rust consumers)

---

1. Executive Summary

System Overview

This specification defines a two-layer, open-source M-Pesa (Safaricom Daraja) state-reconciliation engine and Cashu payment processor written in Rust, designed for native embedding or language-agnostic consumption over gRPC and REST:

1. **`daraja-core` (The Rust State Machine & Language-Agnostic Sidecar Daemon):** A standalone, ACID-compliant Rust library crate and local daemon (`daraja-daemon`) wrapping Safaricom's Daraja APIs: STK Push (`Lipa na M-Pesa Online`), C2B (`RegisterURL`), STK Push Query, B2C v3 Disbursements (to phone numbers), B2B v1 Disbursements (`BusinessBuyGoods` for Merchant Tills and `BusinessPayBill` for PayBills), Account Balance Query, and Transaction Status Query. While implemented in Rust for compile-time state safety and single-binary deployment, `daraja-daemon` exposes a language-agnostic gRPC and HTTP/REST interface alongside an `AGENTS.md` integration skill so external platforms written in TypeScript/Node.js, Python, Go, C#, or PHP can consume idempotent M-Pesa collections and disbursements without writing telecom state machines.
2. **`cdk-daraja-processor` (The Cashu Mint gRPC Plugin):** A Rust gRPC server implementing the `cdk_payment_processor.CdkPaymentProcessor` protobuf service for `cdk-mintd`. It enables Cashu mints to accept M-Pesa deposits to mint Chaumian ecash (`KES` or `sat` units) and disburse M-Pesa via B2C (to phone numbers) or B2B (directly to Merchant Tills and PayBills) when users melt ecash proofs.

Core Architectural Value & Structural Moat

1. **Idempotent Ambiguity Resolution (The Proof-to-Payout Lock):** Connecting an irreversible, cryptographically blind bearer asset (Cashu) to an asynchronous

telecom API (Daraja) creates severe double-spend and insolvency risks. If Daraja returns HTTP 500/503, drops a connection, or fires a `QueueTimeOutURL` callback, a naive adapter that marks the payout as `Failed` will unlock the user's ecash proofs while Safaricom still executes the transfer, draining the mint anonymously. This system enforces an `UNKNOWN_IN_FLIGHT` saga state that keeps ecash proofs locked in `PENDING` state and resolves ambiguity through deterministic `OriginatorConversationID` duplicate-rejection probing (`500.002.1001`) and out-of-band status reconciliation.

2. **Off-Rail Bearer Transfers at Zero Marginal Cost:** Standard M-Pesa transfers incur tariffs on every peer-to-peer hop. By converting an M-Pesa deposit into Chaumian blind-signed bearer tokens (`cashuB...`), value can transfer across unlimited peer-to-peer or API-to-API hops over QR codes, WhatsApp, SMS, or HTTP headers at 0% fee and with mathematical transaction blindness, touching Safaricom's rails only at initial deposit and final settlement.
3. **Consumer MSISDN Privacy & Anti-Spam Shield (B2B Exit):** Standard C2B Till and PayBill payments expose the payer's personal phone number and identity to merchant marketing databases. When melting Cashu tokens to a Till (`BusinessBuyGoods`) or PayBill (`BusinessPayBill`) via `cdk-daraja-processor`, the payment executes over Daraja B2B from the Mint's Corporate Shortcode. The merchant receives guaranteed settlement and the user receives the official M-Pesa `TransactionID` receipt code, while the user's personal phone number remains completely masked from third-party merchants.
4. **Elimination of Aggregator Counterparty Risk & Fees:** Licensed operators and merchants holding their own Safaricom C2B, B2C, and B2B shortcodes can deploy this self-hosted binary directly against Daraja at 0% software fee, eliminating third-party payment aggregator fees and custodial float exposure.
5. **Build Once in Rust, Connect from Any Stack in 60 Seconds:** By compiling to a static ~15MB Rust binary, exposing clean gRPC/REST socket boundaries, and shipping a structured `AGENTS.md` specification for coding agents (Claude Code, Cursor, Codex), the engine serves both Rust-native ecosystems (`cdk-mintd`) and polyglot web backends (TypeScript/Next.js, Python/Django/FastAPI, Go, PHP/Laravel, C#).

---

2. Regulatory & Legal Perimeter (Kenya — October 2026)

Note: This section documents the statutory framework in force in Kenya as of October 2026 for architectural alignment. Operators must obtain independent legal counsel prior to production deployment.

2.1 Open-Source Software Shield vs. Operator Licensing

- **Non-Custodial Software Distribution:** This project is released solely as open-source, self-hosted software infrastructure under MIT/Apache-2.0 licenses. The software authors do not custody funds, hold Daraja keys, operate a shared gateway, or intermediate transactions. Each deploying entity supplies its own Safaricom credentials, corporate shortcodes, and settlement bank accounts.
- **KES-Denominated Mints (National Payment System Act & 2026 Bill):** Under the **National Payment System (NPS) Act (2011)** and the **Draft National Payment System Policy & National Payment System Bill, 2026** (public consultation closed October 9, 2026), **electronic money (e-money)** is defined as monetary value represented by a claim on the issuer, stored electronically, issued on receipt of funds, and accepted as a means of payment by entities other than the issuer. A public mint issuing `KES`-denominated tokens redeemable at third-party merchant Tills or PayBills constitutes e-money issuance and Payment Service Provider (PSP) activity requiring Central Bank of Kenya (CBK) authorization (or admission into the CBK Regulatory Sandbox introduced under the 2026 NPS Bill), unless restricted to a legally exempt closed-loop environment.
- **Crypto/SAT-Denominated Mints & Bridges (VASP Act 2025 & Regulations 2026):** Under the **Virtual Asset Service Providers Act, 2025 (Act No. 20 of 2025)** and the **Virtual Asset Service Providers Regulations, 2026 (Legal Notice No. 134 of 2026, gazetted July 22, 2026; statutory transition deadline November 4, 2026)**, entities operating virtual asset payment processing, conversion, or custody in or from Kenya are subject to dual CBK/CMA supervision and **Part IX / Fifth Schedule** capital requirements:
    - **Virtual Asset Payment Processor (CBK):** Minimum paid-up capital of **KES 10,000,000** (plus liquid capital of KES 4,000,000 or 8% of total liabilities, whichever is higher).
    - **Virtual Asset Broker / Tokenisation Platform (CMA):** Minimum paid-up capital of **KES 10,000,000**.

    - **Virtual Asset Manager / ICO Provider (CMA):** Minimum paid-up capital of **KES 20,000,000**.
    - **Virtual Asset Exchange (CMA):** Minimum paid-up capital of **KES 100,000,000**.
    - **Virtual Asset Wallet Provider (CBK):** Minimum paid-up capital of **KES 150,000,000**.
    - **Stablecoin Issuer (CBK):** Minimum paid-up capital of **KES 300,000,000**.

2.2 Corporate Shortcode Prerequisites

Production access to Safaricom Daraja C2B, B2C, and B2B APIs requires a registered corporate entity (Certificate of Incorporation, CR12, KRA PIN, board resolution, corporate

bank settlement account, and M-Pesa Business Administrator portal vetting). Informal groups cannot obtain direct B2C/B2B shortcodes without operating under a registered corporate or SACCO entity.

3. Target Audience & Scope Prioritization

To maintain strict execution discipline for v1.0, target deployments are separated into immediate primary targets and post-validation integrations:

Tier	Target Segment	Integration Mechanism	Operational Problem Solved	Scope Status
Tier 1 (Primary)	**Cashu Mints, Bitcoin Bridges & Crypto Checkout Off-Ramps** (`cdk-mintd` operators, Bitcoin circular economies, and merchants pairing receive-only crypto checkouts like OpenReceive/BTCPay with M-Pesa settlement)	`cdk-daraja-processor` (gRPC) + `daraja-core` (Rust crate / daemon)	Eliminates third-party aggregator spreads and custodial float risk; prevents double-spend losses on Daraja 503 timeouts; enables direct B2B Till/PayBill off-ramping.	**v1.0 Core Scope**
Tier 2 (Primary)	**Polyglot Kenyan Platforms & SaaS** (TypeScript/Node.js, Python, Go, PHP rental, e-commerce, and split-payout platforms)	`daraja-daemon` (Language-agnostic gRPC / REST sidecar + `AGENTS.md` skill)	Replaces brittle, framework-specific Daraja webhook controllers with a single ACID-compliant Rust reconciliation engine; prevents wrong-account PayBill black holes via synchronous C2B `ValidationURL`.	**v1.0 Core Scope**
Tier 3 (Secondary)	**Closed-Loop Institutional Vouchers, SACCOs & Third-Party Off-Ramps**	Full stack + Custom Wallet UI / CBK Sandbox or PSP license	Provides zero-fee internal bearer transfers and automated B2C/B2B settlement once institutional KYC and wallet UI are in place.	**v1.1 Expansion** (Post-Phase 0 validation)

---

4. Core Technical Decisions & Verified Architecture Matrix

Architectural Domain	Technical Choice	Verified Engineering Rationale
Core Implementation Language	**Rust (Edition 2024)** (`tokio`, `axum`, `tonic`, `sqlx`)	Provides compile-time state machine safety, zero garbage-collection pauses, native compatibility with `cashubtc/cdk` crates, and compiles to a single ~15MB static binary (`musl`).
Polyglot Interoperability	**2-Crate Core + Sidecar Daemon + `AGENTS.md`** (`daraja-core`, `cdk-daraja-processor`, `daraja-daemon`)	Decouples Safaricom telecom reconciliation from Cashu mint logic. External applications in TypeScript, Python, Go, PHP, or C# connect to `daraja-daemon` via gRPC or REST, aided by an `AGENTS.md` prompt spec for automated AI coding agent integration.
CDK Protobuf Contract	**Pinned `cdk_payment_processor.CdkPaymentProcessor` Service**	Implements the exact 7 RPCs defined in `cashubtc/cdk`: `GetSettings`, `CreatePayment`, `GetPaymentQuote`, `MakePayment`, `CheckIncomingPayment`, `CheckOutgoingPayment`, and `WaitIncomingPayment`. Pins the exact `cdk` git tag to prevent breaking changes from upstream protocol version bumps.
NUT-04 / NUT-05 Payment Method & Dual-Rail UX	**PaymentMethod::Custom("mpesa") + Reference Web/CLI Client**	Mainstream mobile wallets (Minibits, Macadamia, eNuts) hardcode `bolt11` UI flows. v1.0 ships a lightweight reference web/CLI client featuring simultaneous STK Push + one-tap C2B PayBill copy fallback and a 35s STK retry

Architectural Domain	Technical Choice	Verified Engineering Rationale
		cooldown synced to `stkpushquery`.
Persistence Engine	**SQLite (WAL Mode via sqlx) + Optional PostgreSQL**	Daraja callbacks are asynchronous and untrusted. SQLite WAL provides zero-config ACID durability for standalone mints, while PostgreSQL supports high-concurrency SaaS deployments.
Credential Isolation	**Dual OAuth Actors (CollectionApp + DisbursementApp)**	Safaricom separates C2B/STK Push shortcodes from B2C/B2B disbursement shortcodes across distinct Daraja apps and permissions. Independent `tokio::sync::RwLock` token actors refresh at 50 minutes (before the 3,600s expiry).
Webhook Security Model	**Zero-Trust: Cryptographic URL Token + Quote Binding + Mandatory Query Confirmation**	Daraja webhooks are unsigned, community CIDR lists (`196.201.212.0/24`, `196.201.213.0/24`, `196.201.214.0/24`) are unofficial/incomplete, and threshold-only checks allow sub-threshold spoofing. Every incoming callback requires a secret URL token, strict quote parameter matching, and mandatory out-of-band query verification regardless of amount.
Disbursement Tariff & Privacy Accounting	**Configurable Corporate Tariff Table + Masked B2B Remitter Identity**	B2C and B2B carry distinct tiered Safaricom corporate tariffs. `GetPaymentQuote` computes the exact tariff from a configurable schedule so the

Architectural Domain	Technical Choice	Verified Engineering Rationale
		mint never loses float on fees. For B2B payouts, the merchant sees only the Mint's Corporate Shortcode/Name as remitter (protecting the user's MSISDN from merchant spam), while the user receives the M-Pesa `TransactionID` receipt as cashier proof.

---

5. Deep-Dive Architecture & State Machines

5.1 Rust Workspace Layout

1. **`crates/daraja-core` (Library Crate):**
    - `auth.rs` — Dual OAuth2 token actors (`CollectionApp` and `DisbursementApp`) with proactive 50-minute refresh and concurrency locks.
    - `crypto.rs` — Base64 STK password generator (`Shortcode + Passkey + EAT Timestamp`) and X.509 `.cer` RSA PKCS#1 v1.5 encryption for B2C and B2B `SecurityCredential`.
    - `webhook.rs` — `axum` HTTP ingress enforcing constant-time URL secret verification (`/callbacks/stk/{secret}`, `/callbacks/c2b/validation/{secret}`, `/callbacks/c2b/confirmation/{secret}`, `/callbacks/b2c/{secret}`, `/callbacks/b2b/{secret}`), optional CIDR filtering (`196.201.212.0/24`, `196.201.213.0/24`, `196.201.214.0/24`), schema validation, and SQLite/Postgres idempotency locking.
    - `reconciler.rs` — Background `tokio` worker executing `stkpushquery/v1/query`, idempotent B2C/B2B retry-probing, `transactionstatus/v1/query` polling, and exception-queue management.
    - `tariffs.rs` — Configurable Safaricom B2C and B2B corporate tariff schedule evaluator.
2. **`crates/cdk-daraja-processor` (Binary & Crate):**
    - `grpc.rs` — `tonic` gRPC server implementing `cdk_payment_processor.CdkPaymentProcessor` (`GetSettings`,

`CreatePayment`, `GetPaymentQuote`, `MakePayment`, `CheckIncomingPayment`, `CheckOutgoingPayment`, `WaitIncomingPayment`).

3. **`crates/daraja-daemon` (Language-Agnostic Sidecar Binary):**
    - Exposes `daraja-core` over a local/networked gRPC and REST/OpenAPI socket so external TypeScript, Python, Go, PHP, or C# applications can initiate collections, disbursements, and balance checks.
4. **`AGENTS.md` (Root Agent Integration Spec):**
    - Self-contained architectural prompt and OpenAPI/Protobuf wiring guide enabling AI coding agents (Claude Code, Cursor, Codex) to integrate `daraja-daemon` idempotent collection and payout workflows into any existing web framework in a single pass.

5.2 Mint Flow (Deposit via STK Push & C2B Fallback) State Machine

1. Quote Creation & Dual-Rail Checkout UX (CreatePayment)

1. Client requests a mint quote for amount `A` (in `KES` or `sat` via pluggable FX quote oracle with a strict expiration TTL).
2. The processor generates an 8-character alphanumeric `account_ref` (e.g., `CDK9F2A1`), inserts a record into `inbound_quotes` with state `ISSUED`, expected `amount = A`, and `shortcode`.
3. If an MSISDN is provided in request options, the processor dispatches `mpesa/stkpush/v1/processrequest`, stores the returned `CheckoutRequestID`, transitions state to `STK_SENT`, and returns a structured dual-rail response containing:
    - 1. Active `CheckoutRequestID` status for the STK Push spinner.
    - 2. Immediate **One-Tap Copy C2B Fallback Instructions** (`PayBill Shortcode` + `account_ref` + exact `Amount`), allowing the user to pay manually from the M-Pesa app or SIM Toolkit (or from a secondary phone number) if GSM signaling drops the STK prompt.
    - 3. A **35-second STK Retry Cooldown timestamp** synced to the first `stkpushquery` poll so the client UI only enables "Resend STK Prompt" once Safaricom's subscriber session lock has cleared.

2. Callback Ingestion & 4-Gate Zero-Trust Verification

When `/callbacks/stk/{secret}` or `/callbacks/c2b/confirmation/{secret}` receives a payload with `ResultCode: 0` and `MpesaReceiptNumber`:

4. **Gate 1 (Cryptographic URL Secret & Quote Binding):** Verify the URL `{secret}` token in constant time. Match `CheckoutRequestID` (for STK) or `BillRefNumber`

`== account_ref` (for C2B) against `inbound_quotes`, and verify `BusinessShortCode` matches the configured collection shortcode.

5. **Gate 2 (ACID Receipt Deduplication):** Insert `MpesaReceiptNumber` into `mpesa_receipts_ledger` (`UNIQUE` constraint). If a conflict occurs, drop the callback immediately to prevent replay attacks.
6. **Gate 3 (Exact Amount Verification):** Verify callback `Amount == expected_amount`. If mismatched or expired, route immediately to the **Exception State Machine** (Section 5.2.4).
7. **Gate 4 (Mandatory Out-of-Band Verification for All Amounts):** Because Daraja callbacks are unsigned, the engine verifies **every** transaction (regardless of KES amount) before minting:
    - **STK Push Path:** Query `mpesa/stkpushquery/v1/query` (using exponential backoff if throttled by Daraja spike arrest `500.001.1001`).
    - **C2B PayBill Path:** Because `stkpushquery` has no visibility into manual C2B PayBill deposits, verify via `mpesa/transactionstatus/v1/query` using `TransactionID = MpesaReceiptNumber` (or via validated C2B Confirmation correlation when synchronous `ValidationURL` pre-check is enabled).

3. Active Polling Loop & Settlement Emission

8. If a quote remains in `STK_SENT` for 35 seconds without a callback, `reconciler.rs` polls `mpesa/stkpushquery/v1/query` at 35s, 65s, and 110s.
9. Upon verification, state transitions to `PAID`, and `WaitIncomingPayment` streams the settled quote identifier to `cdk-mintd` so the user can mint blinded tokens.

4. Missing Money Paths, Pre-Validation & Reversal Defense (unreconciled_deposits)

Manual C2B PayBill payments frequently deviate from quote parameters. To eliminate the common M-Pesa failure mode where users send funds to a mistyped PayBill account number and cannot recover them, `daraja-core` enforces both **synchronous pre-validation** and **post-settlement exception queues**:

1. **Synchronous C2B Typo Rejection (`ValidationURL`):** When a user initiates a manual C2B PayBill transfer, Safaricom calls `/callbacks/c2b/validation/{secret}` before deducting funds. `daraja-core` checks that `BillRefNumber` matches an active `ISSUED` or `STK_SENT` quote (`account_ref`) and that `TransAmount == expected_amount`. If no matching quote exists (e.g., user typo), `daraja-core` returns a non-zero rejection code (`ResultCode: "C2B00012"` Invalid Account Number or `"C2B00013"` Invalid Amount), causing M-Pesa to abort the transaction immediately before money leaves the user's phone.
2. **Underpayment (`Amount < expected_amount`):** If C2B validation is bypassed or times out and an underpayment lands, the quote transitions to `UNDERPAID_HELD`.

No ecash is minted. The user/operator can either: (a) apply a second C2B top-up referencing the same `account_ref` to reach `expected_amount`, or (b) trigger an automated B2C refund of `Amount - b2c_tariff_fee`.

3. **Overpayment (`Amount > expected_amount`):** Quote transitions to `OVERPAID_HELD` (or optionally settles `expected_amount` and records `Amount - expected_amount` as a refundable credit in `unreconciled_deposits`).
4. **Late Deposit (Post-TTL) or Orphan C2B (`BillRefNumber` Not Found):** Recorded as `ORPHAN_UNCLAIMED` in `unreconciled_deposits`. For `KES`-unit quotes (zero FX risk), operators can configure an automatic late-settlement grace window; for `sat`-unit quotes where the FX quote expired, funds remain held for manual refund or re-quoting at current spot rate.
5. **Post-Mint M-Pesa Reversal Defense:** Because minted Chaumian ecash is bearer-portable and anonymous, the mint cannot claw back tokens if a depositor initiates a fraudulent M-Pesa reversal after spending ecash. To mitigate this:
10. Operators must configure their Safaricom C2B shortcode for **Maker-Checker / Organization-Approved Reversals Only** (blocking unilateral customer reversals).
11. Configurable velocity limits and settlement hold windows apply to high-value deposits from unverified MSISDNs.

5.3 Melt Flow (Payout via B2C v3 & B2B v1) & Saga Recovery

1. Destination Parsing, Tariff Reservation & Anti-Spam Privacy (GetPaymentQuote)

The processor parses the melt target and computes the exact Safaricom corporate tariff from `tariffs.rs`:

12. **Phone Number (`2547XXXXXXXX` / `01XXXXXXXX`):** Routes to `mpesa/b2c/v3/paymentrequest` (`CommandID: BusinessPayment`, `PartyB: <phone>`).
13. **Merchant Till (`TILL:<till_number>`):** Routes to `mpesa/b2b/v1/paymentrequest` (`CommandID: BusinessBuyGoods`, `SenderIdentifierType: 4`, `RecieverIdentifierType: 2`).
14. **Merchant PayBill (`PAYBILL:<shortcode>:<account_ref>`):** Routes to `mpesa/b2b/v1/paymentrequest` (`CommandID: BusinessPayBill`, `SenderIdentifierType: 4`, `RecieverIdentifierType: 4`, `AccountReference: <account_ref>`).
15. **Remitter Identity Masking & Cashier Proof:** B2B payouts originate from the Mint's Corporate Shortcode, completely hiding the user's personal phone number from the receiving merchant or SACCO/bank PayBill and preventing SMS spam harvesting. Upon settlement, the processor returns the M-Pesa `TransactionID` (Receipt Number) in the payment proof/preimage field so the user can display the exact M-Pesa receipt code to the merchant or cashier.

2. Pre-Flight Lock & Dispatch (MakePayment)

16. `cdk-mintd` locks the user's ecash proofs in `PENDING` state and invokes `MakePayment`.
17. `daraja-core` generates a deterministic UUID (`OriginatorConversationID`), commits a row to `outbound_payouts` in state `DISPATCHING` prior to network I/O, and sends the B2C v3 or B2B v1 request.
18. Note on `OriginatorConversationID` tracking: `daraja-core` records both the client-supplied `OriginatorConversationID` and the synchronous response `OriginatorConversationID` / `ConversationID` returned by Daraja's proxy to ensure callback correlation succeeds across both API versions.

■ **3. The Ambiguous Timeout Trap & Two-Stage Resolution**

19. **Strict Invariant:** The processor must **NEVER** return `PaymentStatus::Failed` to `cdk-mintd` on an HTTP 500, HTTP 503, network timeout, or Daraja `QueueTimeOutURL` callback. Returning `Failed` unlocks the user's ecash proofs while Safaricom may still execute the payout.
20. Instead, the processor transitions the record to `UNKNOWN_IN_FLIGHT` and returns `PaymentStatus::Pending` to `cdk-mintd`.
21. **Two-Stage Ambiguity Resolver:**
    - **Stage 1 — Idempotent Duplicate-Rejection Probe (500.002.1001):** When an initial HTTP request times out without a synchronous acknowledgment, `reconciler.rs` re-submits the request with the **exact same `OriginatorConversationID`**. Under Daraja's idempotency rules, if the first request reached Safaricom, Daraja rejects the retry with error code `500.002.1001` (`Duplicate OriginatorConversationID` / `Duplicate Detected`), proving the first payout is in flight; if the first request dropped before reaching Daraja, the retry executes safely once. *(Sandbox Gate: Verified on B2C v3; must be integration-verified on B2B v1 during Phase 0 prior to enabling B2B auto-retry).*
    - **Stage 2 — TransactionStatus Polling & Bounded Pending TTL:** For acknowledged in-flight transactions whose result callback dropped, `reconciler.rs` polls `mpesa/transactionstatus/v1/query`. Because `TransactionStatus` is asynchronous and cannot indefinitely block user funds if a transaction never registered, `daraja-core` enforces a configurable `MAX_PENDING_TTL` (e.g., 2 hours automated polling, 12 hours hard cutoff). If unresolved after `MAX_PENDING_TTL`, the payout transitions to `MANUAL_RECONCILIATION_REQUIRED` and triggers an operator alert to verify against the M-Pesa Organization Portal before marking the payout `PAID` (burning proofs) or `FAILED` (unlocking proofs).

6. Operational Liquidity: Solving the "Double-Float" Problem

Safaricom isolates C2B Customer Collection accounts (Utility/MMF) from B2C/B2B Disbursement Working accounts:

22. **Automated Balance Telemetry:** `daraja-core` periodically queries `mpesa/accountbalance/v1/query` and exports Prometheus metrics (`daraja_b2c_utility_balance_kes`, `daraja_c2b_collection_balance_kes`).
23. **Pre-Quote Melt Circuit Breaker:** If `daraja_b2c_utility_balance_kes` falls below `MIN_FLOAT_THRESHOLD + quote_amount`, `GetPaymentQuote` rejects new melt requests upfront (`Disbursement liquidity rebalancing in progress`) rather than locking user proofs and failing mid-flight.
24. **Optional Internal Float Sweep:** For enterprise M-Pesa organizations where C2B and B2C shortcodes are linked under the same Head Office portal hierarchy, `daraja-core` supports an optional automated trigger via `mpesa/b2b/v1/paymentrequest` (`CommandID: BusinessTransferFromMMFToUtility`). For standard unlinked shortcodes, low-float alerts notify the operator to initiate a bank/PesaLink transfer.

---

7. Execution Checklist & 3-Week Build Roadmap

Phase 0: Protocol, Sandbox & Legal Verification (Days 1–3)

- Pin target `cashubtc/cdk` version and compile the `cdk_payment_processor.CdkPaymentProcessor` protobuf definitions (`GetSettings`, `CreatePayment`, `GetPaymentQuote`, `MakePayment`, `CheckIncomingPayment`, `CheckOutgoingPayment`, `WaitIncomingPayment`).
- Verify `OriginatorConversationID` duplicate rejection (`500.002.1001`) behavior across both **B2C v3** and **B2B v1** endpoints in the Safaricom Daraja sandbox/staging environment.
- Validate NUT-04/NUT-05 custom payment method (`mpesa`) handling with `cdk-cli` and scope the minimal reference web/CLI wallet client.
- Confirm operator regulatory classification under the National Payment System Bill 2026 (for KES mints) and VASP Regulations 2026 (for SAT mints/bridges).

Phase 1: daraja-core Rust Engine (Days 4–12)

- Initialize Rust Cargo workspace (`crates/daraja-core`, `crates/cdk-daraja-processor`, `crates/daraja-daemon`).
- Build dual-credential OAuth actors (`CollectionApp` + `DisbursementApp`) with 50-minute proactive refresh and `tokio::sync::RwLock`.

- Implement `sqlx` relational schema (`inbound_quotes`, `outbound_payouts`, `mpesa_receipts_ledger`, `unreconciled_deposits`) with strict `UNIQUE` constraints.
- Build `axum` webhook endpoints with constant-time URL secret verification, synchronous C2B `ValidationURL` typo/amount rejection (`C2B00012` / `C2B00013`), and mandatory out-of-band query confirmation (`stkpushquery` + `transactionstatus`).
- Implement B2C v3 and B2B v1 (`BusinessBuyGoods`, `BusinessPayBill`) dispatchers, configurable corporate tariff tables (`tariffs.rs`), `OriginatorConversationID` retry-probing, and `MAX_PENDING_TTL` escalation.

Phase 2: cdk-daraja-processor gRPC Plugin & Reference Client (Days 13–18)

- Implement the `CdkPaymentProcessor` `tonic` gRPC server mapping CDK Mint and Melt lifecycles to `daraja-core`.
- Build the lightweight reference web/CLI client supporting `mpesa` dual-rail deposits (active STK Push + one-tap C2B PayBill copy fallback + 35s retry cooldown) and `PHONE`, `TILL`, and `PAYBILL` melts with M-Pesa `TransactionID` receipt display.
- Execute fault-injection test suite: unsigned/spoofed webhooks, duplicate `MpesaReceiptNumber` replays, mistyped/underpaid/overpaid/orphan C2B deposits, and HTTP 503 timeouts during B2C/B2B disbursements.

Phase 3: Polyglot Sidecar Daemon, AGENTS.md & Packaging (Days 19–21)

- Expose `daraja-daemon` gRPC/REST endpoints with OpenAPI/Protobuf specs for TypeScript, Python, PHP, and Go consumers.
- Author `AGENTS.md` coding-agent skill so external developers using Claude Code, Cursor, or Codex can auto-integrate `daraja-daemon` into existing web backends.
- Publish crates and operator documentation.